Java Core Concepts

Java is a high-level, object-oriented programming language that follows the principle of "write once, run anywhere." This is achieved through the Java Virtual Machine (JVM), which executes platform-independent bytecode instead of machine-specific instructions. Source code written in Java is first compiled into bytecode by the Java compiler, and the JVM interprets or compiles this bytecode into native machine instructions during execution.

The Java ecosystem consists of three major components: the Java Development Kit (JDK), the Java Runtime Environment (JRE), and the JVM. The JDK provides development tools such as the compiler and debugger. The JRE contains the libraries and runtime environment required to execute Java applications, while the JVM is responsible for loading classes, managing memory, and executing bytecode.

Object-Oriented Programming

Java is built around four fundamental object-oriented principles: encapsulation, inheritance, polymorphism, and abstraction.

Encapsulation combines data and behavior inside a class while restricting direct access to internal fields. Abstraction focuses on exposing only essential functionality while hiding implementation details. Inheritance allows a class to reuse the properties and methods of another class. Polymorphism enables a single interface to represent multiple implementations, allowing the same method call to behave differently depending on the object type.

These principles improve modularity, maintainability, and code reuse in large applications.

Collections Framework

The Java Collections Framework provides reusable data structures for storing and manipulating data efficiently.

An `ArrayList` stores elements in a dynamically resizing array and provides constant-time random access. A `LinkedList` stores elements as nodes connected through pointers, making insertions and deletions efficient but random access slower.

`HashMap` stores key-value pairs using hashing. It provides average-case constant-time insertion, deletion, and lookup. Internally, Java 8 optimizes heavily populated buckets by converting linked lists into balanced trees once a threshold is exceeded.

`TreeMap` maintains keys in sorted order using a Red-Black Tree, resulting in logarithmic search complexity.

Multithreading

Java supports concurrent programming through threads. A thread represents an independent path of execution inside a process.

Threads can be created by extending the `Thread` class or implementing the `Runnable` interface. Modern Java applications commonly use the `ExecutorService` framework to manage thread pools rather than creating threads manually.

Synchronization prevents multiple threads from simultaneously modifying shared data. The `volatile` keyword guarantees visibility of updates across threads but does not provide atomicity. For atomic operations, classes from the `java.util.concurrent.atomic` package are preferred.

Garbage Collection

Java automatically manages memory using garbage collection. Objects are allocated on the heap, and the JVM periodically identifies objects that are no longer reachable and reclaims their memory.

Modern JVM implementations divide the heap into generations because most objects have short lifetimes. The Young Generation is collected frequently, while the Old Generation stores long-lived objects. Different collectors such as G1, ZGC, and Shenandoah optimize pause times for different workloads.

Spring Boot

Spring Boot simplifies enterprise application development by providing auto-configuration and embedded web servers. Instead of manually configuring every component, Spring Boot automatically creates and wires beans based on project dependencies.

Dependency Injection is a core feature of Spring. Rather than creating objects directly using the `new` keyword, dependencies are supplied by the Spring container. This reduces coupling between components and improves testability.

REST APIs built with Spring Boot commonly use annotations such as `@RestController`, `@GetMapping`, and `@PostMapping` to expose HTTP endpoints.